

COMP90050 ADBS

Week 5

Q1 Index types

1. What indices are suitable if a table is frequently used for finding records based on three criteria: a list of users' name, a range of users' birthday and a spatial region covering users' residence?

We have seen some example index structures

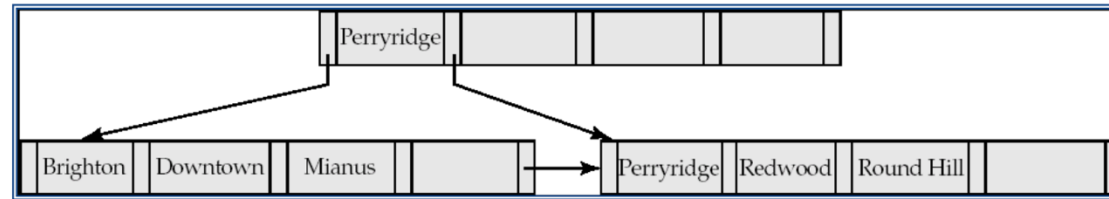
- B+ tree – finding a particular value, finding a range of values
- Hash index – finding a particular value
- Bitmap index – finding total number
- Spatial indexes – range query, nearest neighbor query
 - Quadtree
 - R-tree

- **List of users' name:** Hash index
- **Range of users' birthday:** B+ tree index
- **Region covering users' residence:** spatial index, such as R-tree index and quadtree index

Q2 B+ tree

2. Review the points on indexing with $B+$ trees. Assume a database table has 10,000,000 records and the index is built with a $B+$ tree. The maximum number of children of a node, is denoted as n . How many steps are needed to find a record if $n = 4$? How many steps are needed to find a record if $n = 100$?

Sample tree:
#steps = #nodes we need to access



B⁺-tree for *account* file ($n = 5$)

If there are K search-key values in the file, the height of the tree $\leq \lceil \log_{\lceil n/2 \rceil}(K) \rceil$
 n = maximum number of children of a node

$n = 4$:

Height $\leq \lceil \log_2(10,000,000) \rceil = 24$
24 steps needed

$n = 100$:

Height $\leq \lceil \log_{50}(10,000,000) \rceil = 5$
5 steps needed

Q3 Bitmap indexes

3. Given the database table (shown below), build a bitmap index for the 'State' attribute.

Record Num	Name	State	Income_level
0	John	VIC	L1
1	Diana	NSW	L2
2	Xiaolu	WA	L1
3	Anil	VIC	L4
4	Peter	NSW	L3

1. Bitmap has as many bits as records
2. In a bitmap for value v (e.g. v = VIC):
 - the bit for a record is 1 if the record has the value v (VIC) for the attribute (state)
 - is 0 otherwise

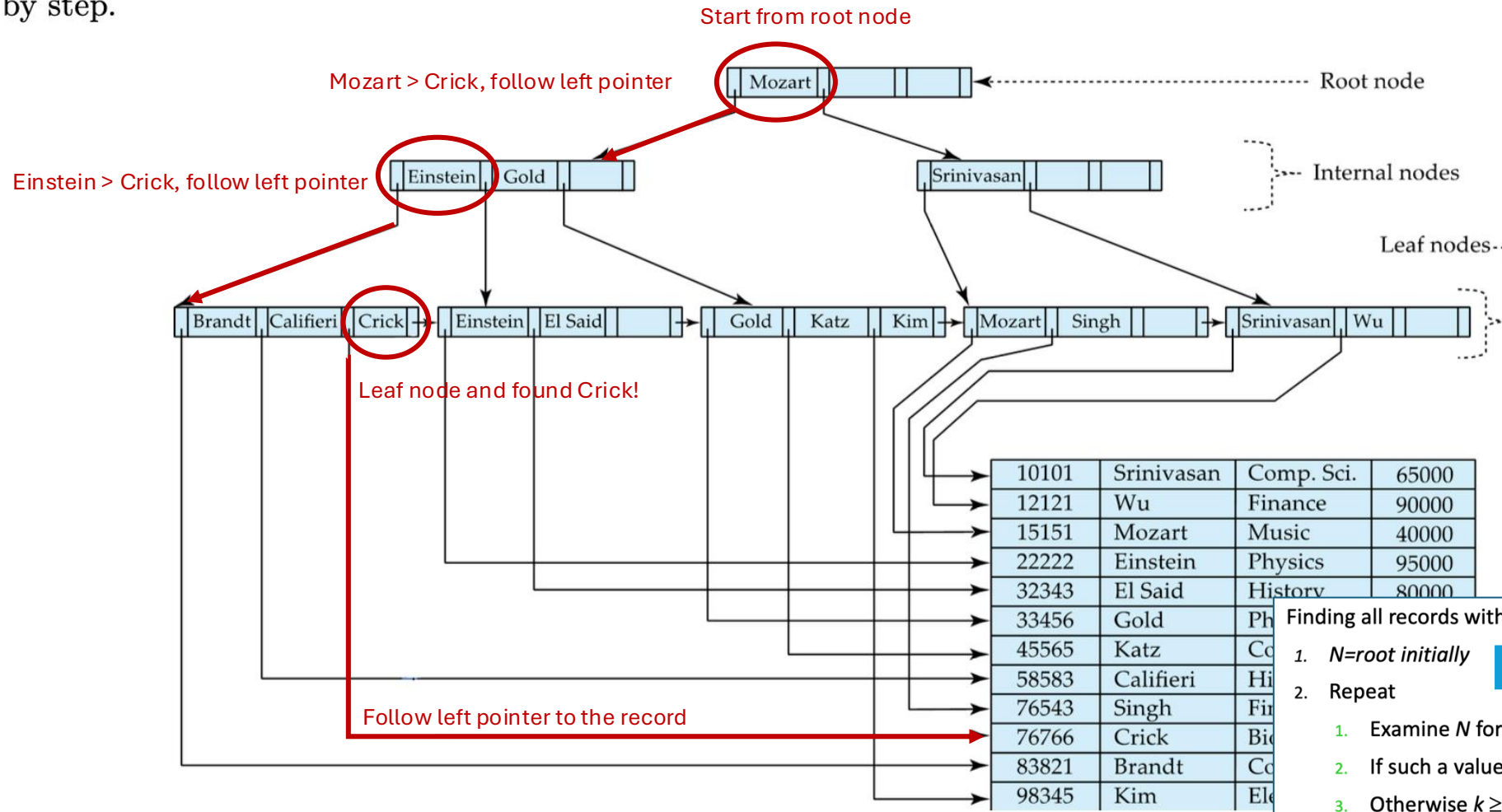
E.g. bitmap for Income_level=L1 is 10100

Bitmap for States:

- VIC : 10010
- NSW : 01001
- WA : 00100

Q4 B+ tree traversal

4. Given the B+ tree example below please search for “Crick” by going through the search algorithm from our lectures step by step.



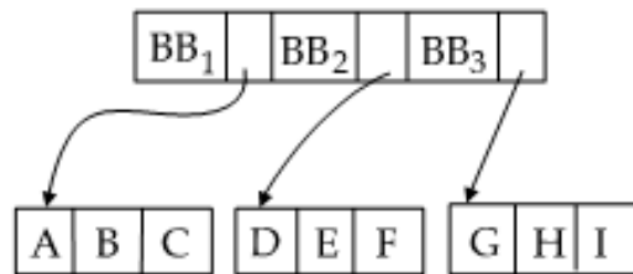
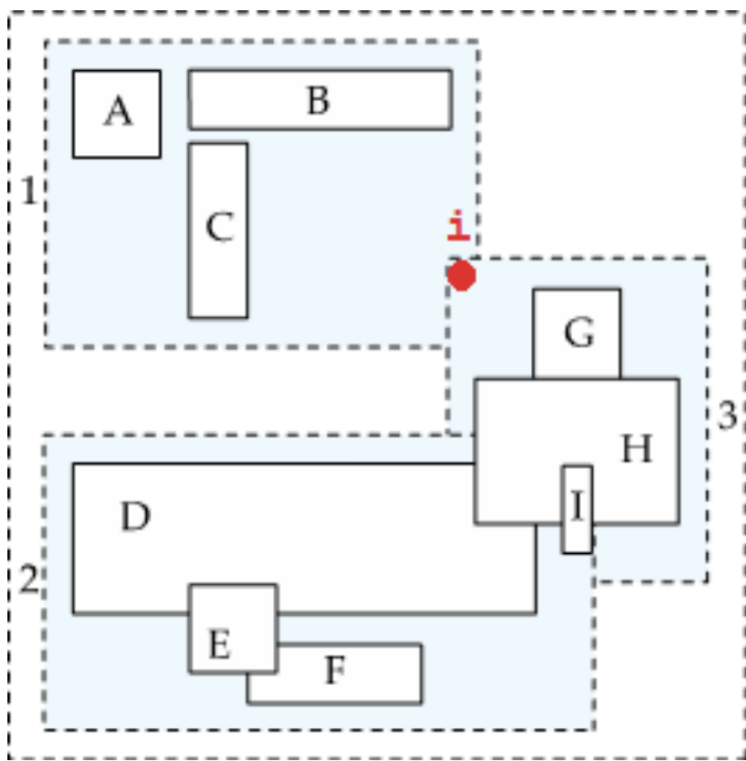
Finding all records with a search-key value of k .

1. $N = \text{root}$ initially
2. Repeat
 1. Examine N for the smallest search-key value $> k$.
 2. If such a value exists, assume it is K_i . Then set $N = P_i$.
 3. Otherwise $k \geq K_{n-1}$. Set $N = P_n$. Follow pointer.Until N is a leaf node
3. If for some i , key $K_i = k$ follow pointer P_i to the desired record or bucket.
4. Else no record with search-key value k exists.

Q5 Nearest Neighbor

5. In the following figure, the rectangles A-I are data objects. These data objects are indexed using an R-tree, where BB1-BB3 denote bounding boxes. Given a query point location denoted as 'i' in the figure, traverse the nodes of the R-tree below in a best-first manner (that is, retrieve the node with the shortest distance from 'i' in each iteration) as discussed in class to find the 1st nearest neighbour of query point "i". If 'i' is inside a bounding box (or rectangle), the distance from i to that bounding box is zero. If there are more than one bounding boxes (or rectangles) with the same distance from i, choose the one alphabetically (e.g., i is inside both BB1 and BB3, choose BB1 first). Is there anything peculiar that you notice while traversing an R-tree?

[Note: Consider the distances just visually].



Nearest neighbor in R-Trees

To find the nearest neighbor of a given query point/region, do the following, starting from the root node:

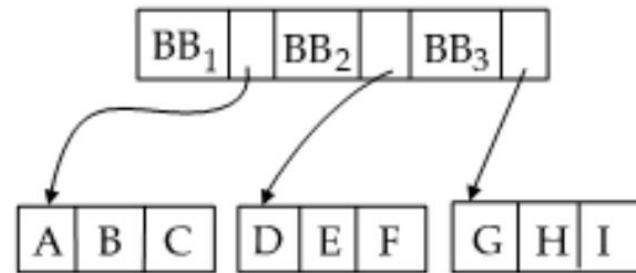
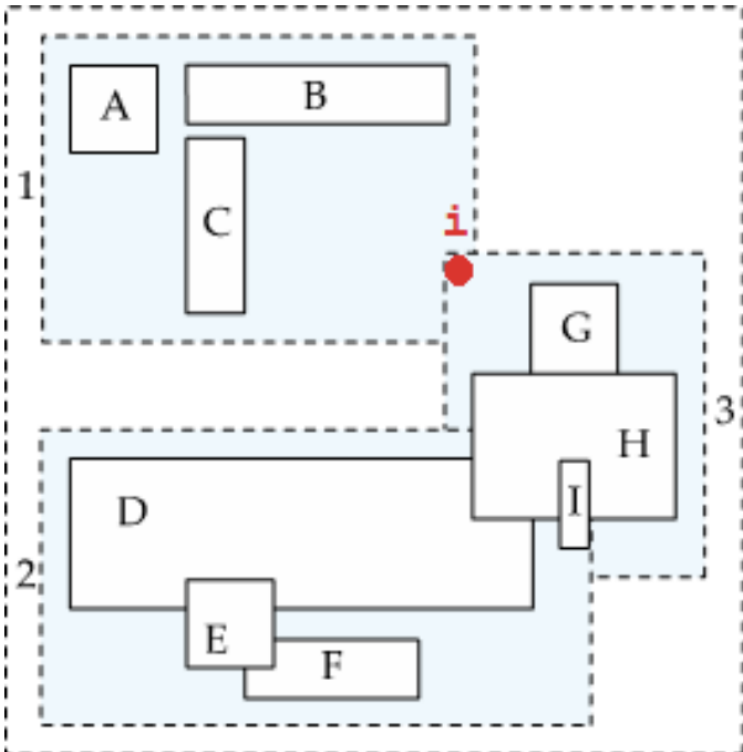
- Use a sorted priority queue of the R-tree nodes based on the minimum distance from the query
- Traverse the node that is in the top of the priority queue, and put its elements in the queue. Continue
- Stop when the top node is a data object (first NN has been found)
- To find k number of NN, continue the above steps until k number data objects are found as the top node of the queue.

This algorithm is a best-first search algorithm

Q5 Nearest Neighbor

5. In the following figure, the rectangles A-I are data objects. These data objects are indexed using an R-tree, where BB1-BB3 denote bounding boxes. Given a query point location denoted as 'i' in the figure, traverse the nodes of the R-tree below in a best-first manner (that is, retrieve the node with the shortest distance from 'i' in each iteration) as discussed in class to find the 1st nearest neighbour of query point "i". If 'i' is inside a bounding box (or rectangle), the distance from i to that bounding box is zero. If there are more than one bounding boxes (or rectangles) with the same distance from i, choose the one alphabetically (e.g., i is inside both BB1 and BB3, choose BB1 first). Is there anything peculiar that you notice while traversing an R-tree?

[Note: Consider the distances just visually].



1. Starting from root node elements **BB1, BB3, BB2**
2. Remove BB1 put A B C into queue **BB3, B, BB2, C, A**
3. Remove BB3 and put G H I into queue **G, H, B, BB2, C, I, A**
4. First element is a data object. **G is closest.**

(the exact ordering of the later ones are not important for this example, as exact distances are not given)

- Overlaps in the bounding boxes lead to the need to traverse multiple branches.
- We are unsure of the position of objects inside bounding box element before traversing it, this may necessitate investigating multiple bounding boxes